

Índice

1. Modelos de datos: perspectiva de datos	2
1.1. Concepto y finalidad de los modelos de datos	2
1.2. Entidades, atributos, identificadores y relaciones.....	2
1.3. Cardinalidad, restricciones e integridad de los datos	3
1.4. Diagrama Entidad–Relación	3
1.5. Diccionario de datos	3
1.6. Importancia de la perspectiva de datos en la Ingeniería de Requisitos.....	4
2. Modelos funcionales: perspectiva funcional	5
2.1. Concepto y propósito de los modelos funcionales	5
2.2. Requisitos funcionales y funciones del sistema	5
2.3. Casos de uso.....	6
2.4. Diagramas de actividades	6
2.5. Diagramas de flujo de datos.....	6
2.6. Aporte de la perspectiva funcional a la especificación de requisitos	7
3. Modelos de comportamiento: perspectiva de comportamiento	8
3.1. Concepto y propósito de los modelos de comportamiento.....	8
3.2. Comportamiento dinámico del sistema.....	8
3.3. Diagramas de estados	9
3.4. Diagramas de secuencia	9
3.5. Diagramas de comunicación.....	9
3.6. Importancia de la perspectiva de comportamiento en la validación de requisitos	10
4. Interrelaciones entre las tres perspectivas	11
4.1. Relación entre datos, funciones y comportamiento	11
4.2. Trazabilidad entre los modelos.....	11
4.3. Consistencia entre las perspectivas.....	11
4.4. Ejemplo integrado	12
4.5. Importancia de utilizar las tres perspectivas conjuntamente	12

1. Modelos de datos: perspectiva de datos

1.1. Concepto y finalidad de los modelos de datos

Los modelos de datos constituyen una representación conceptual de la información que un sistema de software debe almacenar, procesar y administrar durante su operación. En el contexto de la Ingeniería de Requisitos, estos modelos permiten describir la estructura lógica de los datos antes de iniciar el diseño de la base de datos o el desarrollo del sistema, facilitando la comprensión de los requisitos relacionados con la información [1]. La especificación de requisitos debe identificar claramente los datos que intervienen en cada proceso del sistema, ya que estos forman parte esencial de la documentación que guía las etapas posteriores del ciclo de vida del software [2].

Desde una perspectiva de análisis, los modelos de datos ayudan a transformar las necesidades del usuario en estructuras organizadas que posteriormente serán implementadas mediante tablas, objetos u otros mecanismos de almacenamiento. El modelado de datos reduce las ambigüedades durante la ingeniería de requisitos y facilita la comunicación entre clientes, analistas y desarrolladores, debido a que todos trabajan sobre una misma representación conceptual de la información [3]. Asimismo, el modelado de datos constituye una actividad esencial dentro del análisis del software, ya que permite comprender el dominio del problema antes de definir procesos o diseñar componentes, disminuyendo significativamente la probabilidad de errores durante la implementación [2], [4].

El objetivo principal de un modelo de datos consiste en representar de manera organizada la información que será utilizada por el sistema, estableciendo qué datos existen, cuáles son sus características y cómo se relacionan entre sí. Un modelo correctamente elaborado permite garantizar la integridad, consistencia y disponibilidad de la información durante todo el ciclo de vida del software [5]. Entre los objetivos destacan: identificar la información necesaria, organizar los datos de forma estructurada, reducir redundancias e inconsistencias, facilitar el diseño de bases de datos, mejorar la comunicación entre usuarios y desarrolladores, y servir como base para el diseño lógico y físico del sistema.

La correcta definición de la información constituye un elemento fundamental para garantizar la calidad de los productos desarrollados, ya que los datos representan uno de los activos más importantes de cualquier sistema de información [6].

1.2. Entidades, atributos, identificadores y relaciones

Todo modelo de datos está conformado por tres elementos fundamentales: entidades, atributos y relaciones [5], [7].

Las entidades representan objetos, personas, lugares, eventos o conceptos del mundo real acerca de los cuales el sistema necesita almacenar información. Cada entidad posee identidad propia y puede ser identificada de manera única dentro del modelo. La correcta

identificación de entidades constituye uno de los primeros pasos del análisis de requisitos, ya que permite definir el alcance de la información que administrará el sistema [4].

Los atributos representan las características o propiedades de una entidad. Cada atributo almacena información específica que describe un objeto determinado. Una adecuada definición de atributos permite evitar duplicidad de información y facilita la normalización de la base de datos [5].

Las relaciones describen la asociación existente entre dos o más entidades. Estas asociaciones permiten representar las reglas del negocio y constituyen la base para definir las cardinalidades, fundamentales durante el diseño de bases de datos relacionales [5], [7].

1.3. Cardinalidad, restricciones e integridad de los datos

La cardinalidad define el número de instancias de una entidad que pueden participar en una relación. Los tipos más comunes son uno a uno (1:1), uno a muchos (1:N) y muchos a muchos (N:M) [5]. Las restricciones de integridad, como la integridad de entidad y la integridad referencial, garantizan que los datos sean consistentes y válidos. La clave primaria identifica de manera única cada instancia de una entidad, mientras que las claves foráneas establecen los vínculos entre entidades y aseguran que no existan referencias a datos inexistentes [4].

1.4. Diagrama Entidad–Relación

El Diagrama Entidad–Relación (DER) es uno de los modelos conceptuales más utilizados para representar gráficamente la estructura de los datos de un sistema. Propuesto por Chen [7], permite representar visualmente las entidades, sus atributos, las relaciones, las cardinalidades y las claves primarias y foráneas. El principal beneficio del DER consiste en facilitar la comunicación entre usuarios y desarrolladores, ya que proporciona una representación sencilla del dominio del problema antes de comenzar la implementación física de la base de datos. Además, el DER permite detectar errores tempranos relacionados con información duplicada, relaciones incorrectas o ausencia de entidades importantes, reduciendo costos durante las etapas posteriores del desarrollo [3], [5].

1.5. Diccionario de datos

El diccionario de datos constituye un documento complementario al DER donde se describe detalladamente cada uno de los elementos que forman parte de la base de datos. Generalmente incluye información como: nombre del atributo, descripción, tipo de dato, longitud, restricciones, valores permitidos, clave primaria, clave foránea y reglas de validación [4]. Este documento facilita la comprensión del sistema, mejora la trazabilidad de los requisitos y sirve como referencia durante las actividades de mantenimiento del

software. El diccionario de datos reduce la ambigüedad entre los miembros del equipo y favorece la estandarización de la información utilizada durante el desarrollo [2], [8].

1.6. Importancia de la perspectiva de datos en la Ingeniería de Requisitos

Los modelos de datos constituyen uno de los pilares fundamentales de la Ingeniería de Requisitos porque permiten comprender la información que será manipulada por el sistema antes de definir los procesos funcionales o el comportamiento dinámico de la aplicación. Una especificación incompleta de los datos puede ocasionar información redundante, inconsistencias, errores durante el diseño, baja calidad de la base de datos y dificultades de mantenimiento. Por el contrario, un modelo de datos correctamente elaborado facilita la validación de requisitos, mejora la comunicación entre las partes interesadas y proporciona una base sólida para las siguientes etapas del desarrollo [4], [8]. Una adecuada gestión de la información contribuye directamente a características como la mantenibilidad, confiabilidad, integridad y eficiencia del sistema [9].

2. Modelos funcionales: perspectiva funcional

2.1. Concepto y propósito de los modelos funcionales

Los modelos funcionales constituyen una representación de las funciones, procesos y servicios que un sistema de software debe proporcionar para satisfacer las necesidades de los usuarios. A diferencia de los modelos de datos, que describen la información administrada por el sistema, los modelos funcionales se enfocan en las actividades que el software realiza y en la manera en que los usuarios interactúan con dichas funcionalidades. La especificación de requisitos debe describir claramente las funciones esperadas del sistema, indicando qué acciones debe ejecutar, bajo qué condiciones y cuáles serán los resultados esperados [1].

Durante la Ingeniería de Requisitos, los modelos funcionales permiten transformar las necesidades expresadas por los usuarios en procesos claramente definidos, facilitando la comunicación entre clientes, analistas y desarrolladores. Además, constituyen una herramienta esencial para verificar que todas las funcionalidades solicitadas hayan sido identificadas y documentadas antes de iniciar el diseño o la implementación del software [3], [10]. El modelado funcional representa uno de los principales mecanismos para comprender el comportamiento operativo del sistema, ya que permite identificar las actividades que generan valor para el usuario y la forma en que estas se relacionan con los objetivos del negocio [2], [4].

El principal objetivo de los modelos funcionales consiste en representar de manera organizada todas las funciones que debe realizar un sistema para cumplir con los requisitos establecidos por el cliente. Entre los objetivos más importantes se encuentran: identificar las funciones principales del sistema, describir los procesos de negocio, facilitar la comunicación entre usuarios y desarrolladores, detectar funciones faltantes o redundantes, servir como base para el diseño y las pruebas del software, y mejorar la comprensión de los requisitos funcionales [8]. La correcta elaboración de modelos funcionales permite validar tempranamente el alcance del sistema y reducir el riesgo de desarrollar funcionalidades innecesarias o de omitir procesos importantes.

2.2. Requisitos funcionales y funciones del sistema

Los requisitos funcionales especifican qué debe hacer el sistema, es decir, describen las interacciones entre el sistema y su entorno [10]. Estos requisitos se derivan de las necesidades de los usuarios y se expresan en términos de servicios, tareas o comportamientos que el software debe ofrecer [8]. La identificación y documentación de estos requisitos es fundamental para establecer el alcance del proyecto y para guiar las fases de diseño e implementación [4].

2.3. Casos de uso

Uno de los modelos funcionales más utilizados en Ingeniería de Software son los casos de uso. Un caso de uso describe la interacción entre un actor y el sistema para lograr un objetivo específico, mostrando las acciones que debe ejecutar el software para satisfacer una necesidad del usuario. Cada caso de uso representa un escenario de utilización del sistema y generalmente incluye: nombre, objetivo, actor principal, actores secundarios, precondiciones, flujo principal de eventos, flujos alternativos y postcondiciones [8], [11].

Los casos de uso permiten comprender fácilmente las funcionalidades del sistema desde el punto de vista del usuario, convirtiéndose en una herramienta fundamental para la validación de requisitos y la comunicación entre todas las partes interesadas. Además, sirven como base para la elaboración de diagramas UML y el diseño de casos de prueba durante las etapas posteriores del desarrollo [4], [11].

2.4. Diagramas de actividades

Los diagramas de actividades pertenecen al lenguaje UML [12] y permiten representar gráficamente el flujo de ejecución de un proceso mediante acciones, decisiones, bifurcaciones y sincronizaciones. Su principal finalidad consiste en describir la secuencia lógica de actividades que realiza el sistema para completar una determinada funcionalidad [13]. Estos diagramas están conformados por elementos como nodo inicial, actividades, decisiones, flujos de control, barras de sincronización y nodo final [12]. Una de sus principales ventajas consiste en que permiten visualizar procesos complejos de manera sencilla, facilitando la identificación de cuellos de botella, actividades redundantes y posibles mejoras antes de implementar el sistema [8].

Los diagramas de actividades permiten representar claramente los procesos del negocio y constituyen un complemento importante para los casos de uso, ya que muestran el flujo interno de cada funcionalidad [3], [13].

2.5. Diagramas de flujo de datos

En algunas metodologías tradicionales de análisis de sistemas todavía se utilizan los Diagramas de Flujo de Datos (DFD) para representar la circulación de la información dentro del sistema. Aunque UML ha adquirido mayor protagonismo durante los últimos años, los DFD continúan siendo una herramienta útil para comprender cómo viajan los datos entre procesos, entidades externas y almacenes de información [4]. Los elementos principales de un DFD son: procesos, flujos de datos, entidades externas y almacenes de datos. Los DFD permiten representar únicamente el movimiento de la información, sin preocuparse por aspectos relacionados con la implementación tecnológica. Esto facilita la identificación de entradas, salidas y transformaciones de los datos durante la ejecución de

los procesos [8].

2.6. Aporte de la perspectiva funcional a la especificación de requisitos

La perspectiva funcional constituye uno de los pilares de la especificación de requisitos, ya que permite describir con precisión el comportamiento esperado del sistema desde el punto de vista del usuario. Su utilización facilita la identificación de procesos críticos, mejora la comunicación entre las partes interesadas y reduce el riesgo de desarrollar funcionalidades que no aporten valor al negocio [10]. Además, estos modelos permiten establecer una relación directa entre los requisitos funcionales y los componentes que serán implementados durante el desarrollo del software. Esto favorece la trazabilidad, simplifica la elaboración de pruebas y contribuye al aseguramiento de la calidad del producto final [4]. Los requisitos funcionales deben ser completos, consistentes, verificables y libres de ambigüedades, características que pueden alcanzarse mediante una adecuada utilización de modelos funcionales [1].

3. Modelos de comportamiento: perspectiva de comportamiento

3.1. Concepto y propósito de los modelos de comportamiento

Los modelos de comportamiento representan el funcionamiento dinámico de un sistema de software, describiendo cómo reaccionan sus componentes ante diferentes eventos y cómo evolucionan a lo largo del tiempo. Mientras que los modelos de datos se enfocan en la información administrada y los modelos funcionales describen las funciones del sistema, los modelos de comportamiento explican la manera en que dichas funciones se ejecutan y cómo interactúan los diferentes elementos durante la operación del software [8]. La especificación de requisitos debe describir no solo las funcionalidades esperadas, sino también el comportamiento del sistema frente a distintas condiciones de operación, garantizando que los requisitos sean completos, verificables y consistentes [1].

En Ingeniería de Software, los modelos de comportamiento son fundamentales porque permiten representar la respuesta del sistema ante acciones de los usuarios, eventos internos o externos y cambios de estado de los objetos. Gracias a estos modelos es posible validar la lógica del sistema antes de iniciar la etapa de implementación, reduciendo errores y facilitando la comunicación entre analistas, diseñadores y desarrolladores [3], [4]. El modelado del comportamiento constituye una práctica esencial durante el análisis y diseño de software, ya que proporciona una visión clara de las interacciones entre los diferentes componentes y facilita la identificación de escenarios excepcionales que podrían afectar el funcionamiento del sistema [2], [13].

El principal objetivo de los modelos de comportamiento es representar la evolución dinámica del sistema durante su ejecución, mostrando cómo responden los objetos y procesos frente a diferentes estímulos o eventos. Estos modelos permiten comprender el flujo temporal de las operaciones y verificar que el comportamiento esperado coincida con los requisitos definidos por los usuarios [10].

3.2. Comportamiento dinámico del sistema

El comportamiento dinámico se refiere a la forma en que el sistema cambia de estado en respuesta a estímulos externos o internos. Comprender este comportamiento es crucial para sistemas que manejan múltiples eventos concurrentes o que tienen ciclos de vida complejos [8]. Los modelos de comportamiento permiten describir cómo evoluciona el sistema desde que un usuario inicia una operación hasta que esta finaliza, identificando los eventos que desencadenan determinadas acciones, las respuestas generadas por el software y los cambios de estado que experimentan los diferentes objetos [4].

3.3. Diagramas de estados

Uno de los modelos de comportamiento más utilizados es el diagrama de estados, perteneciente a UML [12]. Este diagrama representa los diferentes estados por los que puede pasar un objeto durante su ciclo de vida y las transiciones que ocurren como consecuencia de determinados eventos. Cada estado representa una condición específica del objeto, mientras que las transiciones indican el cambio de un estado a otro cuando se cumple una determinada condición o se produce un evento [8], [13].

Los diagramas de estados permiten identificar claramente el comportamiento de los objetos y verificar que todas las posibles transiciones hayan sido consideradas durante el diseño del sistema, especialmente en aplicaciones con múltiples condiciones de operación [4].

3.4. Diagramas de secuencia

Los diagramas de secuencia son otro de los modelos fundamentales de comportamiento definidos por UML [12]. Su finalidad consiste en representar el intercambio de mensajes entre objetos siguiendo un orden cronológico, permitiendo visualizar cómo colaboran los diferentes componentes del sistema para ejecutar una determinada funcionalidad [13]. En este tipo de diagramas participan actores, objetos, líneas de vida, mensajes, activaciones y respuestas [12].

Este tipo de diagramas facilita la comprensión de la lógica interna del sistema y constituye una herramienta muy útil para validar escenarios específicos antes de iniciar la programación [3], [8].

3.5. Diagramas de comunicación

Los diagramas de comunicación, anteriormente conocidos como diagramas de colaboración, también forman parte de UML [12] y representan la interacción entre los objetos del sistema desde una perspectiva estructural. A diferencia de los diagramas de secuencia, que priorizan el orden temporal de los mensajes, los diagramas de comunicación destacan las relaciones existentes entre los objetos y la forma en que colaboran para ejecutar una funcionalidad. Cada objeto aparece conectado mediante enlaces de comunicación y los mensajes se identifican mediante una numeración que indica el orden de ejecución [13]. Esta representación resulta útil cuando se desea comprender la arquitectura de colaboración entre componentes sin enfatizar la línea temporal de las operaciones [8].

3.6. Importancia de la perspectiva de comportamiento en la validación de requisitos

La perspectiva de comportamiento complementa las perspectivas de datos y funcional, proporcionando una visión dinámica del sistema y permitiendo comprender cómo interactúan los diferentes componentes durante la ejecución de los procesos. Gracias a estos modelos es posible detectar errores de lógica, validar secuencias de operación y mejorar la calidad de la especificación de requisitos [10]. Asimismo, los modelos de comportamiento favorecen la trazabilidad entre los requisitos funcionales y los casos de prueba, ya que describen de forma detallada las respuestas esperadas del sistema frente a diferentes situaciones. Esto facilita la construcción de software más confiable, mantenible y alineado con las necesidades del usuario [4]. La documentación clara del comportamiento del sistema garantiza que los requisitos sean verificables y que puedan utilizarse como base para las actividades de diseño, implementación y pruebas [1].

4. Interrelaciones entre las tres perspectivas

4.1. Relación entre datos, funciones y comportamiento

Durante el proceso de Ingeniería de Requisitos, comprender completamente un sistema de software requiere analizarlo desde diferentes puntos de vista. Cada perspectiva aporta información específica sobre el sistema y, en conjunto, permiten construir una especificación de requisitos completa, consistente y libre de ambigüedades. La documentación de requisitos debe proporcionar múltiples vistas del sistema para facilitar la comunicación entre las partes interesadas y garantizar la trazabilidad durante todo el ciclo de vida del software [1], [10].

Las tres perspectivas más utilizadas son la perspectiva de datos, la perspectiva funcional y la perspectiva de comportamiento. Aunque cada una analiza un aspecto diferente del sistema, ninguna resulta suficiente por sí sola. La integración de estas perspectivas permite comprender no solo qué información administra el software, sino también qué funciones realiza y cómo se comporta durante su ejecución [4], [8].

La perspectiva de datos se enfoca en la información que será administrada por el sistema, identificando entidades, atributos y relaciones [5]. La perspectiva funcional describe las operaciones, procesos y servicios que el sistema debe proporcionar [11]. La perspectiva de comportamiento describe la forma en que el sistema responde durante su ejecución, incluyendo cambios de estado e interacciones entre objetos [12].

4.2. Trazabilidad entre los modelos

La trazabilidad es la capacidad de rastrear los requisitos a lo largo del ciclo de vida del software. La integración de las tres perspectivas facilita la trazabilidad porque cada modelo está vinculado a los demás: los elementos de datos se utilizan en las funciones, y las funciones se ejecutan mediante comportamientos específicos [3], [10]. Los modelos de datos proporcionan la estructura de información que requieren los procesos funcionales; los modelos funcionales definen las operaciones que transforman esos datos; y los modelos de comportamiento especifican la secuencia y las condiciones bajo las cuales se ejecutan dichas operaciones [8].

4.3. Consistencia entre las perspectivas

La consistencia entre las perspectivas garantiza que no existan contradicciones entre los modelos. Por ejemplo, los estados definidos en un diagrama de estados deben corresponder a valores posibles de atributos en el modelo de datos, y las operaciones descritas en los casos de uso deben ser coherentes con las transiciones de estado [4]. La verificación de consistencia se realiza mediante revisiones y validaciones cruzadas de los modelos, lo que

permite detectar incoherencias antes de la implementación [2].

4.4. Ejemplo integrado

A modo de ilustración conceptual, se puede considerar un proceso típico de negocio que involucra la gestión de recursos. Desde la perspectiva de datos, se identifican las entidades y relaciones necesarias para almacenar la información relevante. Desde la funcional, se definen los casos de uso que describen las interacciones de los actores con el sistema para completar el proceso. Desde la de comportamiento, se especifican los diagramas de estados y secuencia que muestran la evolución de los objetos y la colaboración entre componentes durante la ejecución del proceso. Esta integración demuestra que las tres perspectivas representan diferentes niveles de análisis sobre un mismo proceso y que únicamente su utilización conjunta permite obtener una especificación de requisitos completa [3], [8].

4.5. Importancia de utilizar las tres perspectivas conjuntamente

La utilización simultánea de las perspectivas de datos, funcional y comportamiento constituye una de las mejores prácticas recomendadas por la Ingeniería de Requisitos moderna. La documentación de requisitos debe proporcionar diferentes vistas del sistema para facilitar la comunicación entre los interesados, mejorar la trazabilidad y reducir la ambigüedad durante el desarrollo [1], [10]. Además, la integración de estas perspectivas aporta múltiples beneficios: mejora la comprensión del dominio del problema, reduce inconsistencias, facilita la comunicación entre clientes y desarrolladores, incrementa la trazabilidad, favorece el diseño de pruebas, disminuye el costo de corrección de errores en etapas posteriores y mejora la calidad del software desarrollado [4], [8]. Una adecuada especificación de requisitos influye directamente en características de calidad como la mantenibilidad, la confiabilidad, la eficiencia y la compatibilidad del software [9].

Las perspectivas de datos, funcional y comportamiento no deben entenderse como modelos independientes, sino como elementos complementarios de una misma especificación de requisitos. Su integración permite construir una representación completa del sistema, proporcionando una base sólida para el diseño, la implementación, las pruebas y el mantenimiento del software. La aplicación conjunta de estas perspectivas, respaldada por normas internacionales como ISO/IEC/IEEE 29148:2018, contribuye significativamente al desarrollo de sistemas de mayor calidad, alineados con las necesidades del usuario y con las buenas prácticas de la Ingeniería de Software [1], [2].

Referencias

- [1] ISO/IEC/IEEE, 29148:2018 — *Requirements engineering*, oct. de 2018. doi: [10.1109/IEEESTD.2018.8559686](https://doi.org/10.1109/IEEESTD.2018.8559686)

- [2] H. Washizaki, *SWEBOK Guide v4.0*, H. Washizaki, ed. IEEE Computer Society, 2024.
- [3] K. Pohl, *Requirements engineering : fundamentals, principles, and techniques*. Springer, 2025, pág. 814.
- [4] R. S. Pressman y B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th. McGraw-Hill Education, 2020.
- [5] R. Elmasri y S. B. Navathe, *Fundamentals of Database Systems*, 7th. Pearson, 2021.
- [6] ISO/IEC/IEEE, *15288:2023 — System life cycle processes*, 2023.
- [7] P. P. Chen, «The Entity-Relationship Model—Toward a Unified View of Data,» *ACM Transactions on Database Systems*, vol. 1, n.º 1, págs. 9-36, 1976. doi: [10.1145/320434.320440](https://doi.org/10.1145/320434.320440)
- [8] I. Sommerville, *Software Engineering*, 10th. Pearson, 2020.
- [9] ISO/IEC, *25010:2023 Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*, 2023.
- [10] S. Wagner, D. M. Fernández, M. Kalinowski y M. Felderer, «Agile requirements engineering in practice: Status quo and critical problems,» *CLEI Electronic Journal*, vol. 21, n.º 1, pág. 3, 2018. doi: [10.19153/cleiej.21.1.3](https://doi.org/10.19153/cleiej.21.1.3)
- [11] M. Luckey y G. Engels, «A systematic literature review of use case specifications research,» *Information and Software Technology*, vol. 126, pág. 106 346, 2020. doi: [10.1016/j.infsof.2020.106346](https://doi.org/10.1016/j.infsof.2020.106346)
- [12] O. M. Group, *Unified Modeling Language (UML) Version 2.5.1 Specification*, 2017. doi: [10.1109/ICSE-SEIP.2017.28](https://doi.org/10.1109/ICSE-SEIP.2017.28)
- [13] C. F. J. Lange, M. R. V. Chaudron y J. Muskens, «UML model consistency: A systematic literature review,» *Journal of Systems and Software*, vol. 172, pág. 110 869, 2021. doi: [10.1016/j.jss.2020.110869](https://doi.org/10.1016/j.jss.2020.110869)